

System Design Basics

Lesson 8: Putting It All Together: Design a Mini URL Shortener

Putting It All Together: Design a Mini URL Shortener – Study Notes

Key Concepts

1. **Scalable Architecture** – How to break the system into independent services (API, storage, caching, background workers) that can grow horizontally.
2. **Low Latency Data Path** – Choosing the right data structures (hash tables, key value stores) and caching layers to serve redirects in < /10 /ms.
3. **Unique ID Generation** – Techniques (base 62 encoding, Snowflake, hashids, deterministic vs. random) that produce short, collision free slugs.
4. **Data Consistency & Partitioning** – Sharding strategies, eventual consistency vs. strong consistency, and handling hot keys.
5. **Operational Concerns** – Monitoring, rate limiting, abuse prevention, and graceful degradation under traffic spikes.

Summary

Designing a URL shortener is an excellent “first project” for learning how to build a distributed, low latency service. The core workflow is simple: a client sends a long URL, the service returns a short alias (the *slug*), and later users request that alias and are redirected to the original URL. Despite the simplicity of the user facing API, the system must handle massive read heavy traffic, guarantee that each slug is unique, and stay responsive even when a single node fails.

To meet these goals we typically separate concerns into three layers:

1. **API Layer** – Stateless HTTP servers (or serverless functions) that validate input, invoke the ID generator, write the mapping, and return the slug.
2. **Storage Layer** – A fast key value store (e.g., Redis, DynamoDB, Cassandra) that holds ``<slug> !<br><long URL>`` pairs. Writes are persisted to durable storage (e.g., S3, relational DB) for backup and analytics.
3. **Cache & Edge Layer** – A CDN or reverse proxy cache (e.g., CloudFront, Cloudflare) that stores the most popular redirects close to the user, cutting latency to a few milliseconds.

By using a **deterministic, base 62 encoded counter** (or a Snowflake style generator) we can produce short, collision free slugs without expensive look ups. Sharding the key value store by slug hash spreads load evenly and prevents hot spotting. Adding a **read through cache** ensures that the majority of redirect traffic hits memory rather than disk, keeping latency low. Finally, operational practices—rate limiting, abuse detection, health checks, and automated scaling—keep the service reliable as traffic grows.

Important Points

- **Stateless API** !' easy horizontal scaling; store no session data.
- **Slug length**: 6–8 characters in base 62 gives ~56 /billion unique URLs; adjust length based on expected scale.
- **ID Generation Options**
 - **Sequential counter** + base 62 (simple, predictable).
 - **Random token** (e.g., 6 byte crypto random) !' requires collision check.
 - **Snowflake** (timestamp + machine ID + sequence) !' sortable, globally unique.
- **Primary data store** should support O(1) reads/writes and be able to scale out (e.g., DynamoDB, Cassandra, Redis Cluster).
- **Cache hierarchy**: CDN !' edge cache !' in process LRU !' persistent KV store.
- **Write path**: API !' generate slug !' write to KV store (primary) !' async replicate to backup storage (e.g., S3, RDS).
- **Read path**: CDN/edge !' API !' KV store (cache miss) !' return long URL !' issue 301/302 redirect.
- **Sharding**: hash slug !' determine partition; keep partition key short to avoid hot partitions.
- **Rate limiting**: per IP or per API key throttling to mitigate abuse (spam, phishing).
- **Monitoring**: latency (p99), error rate, cache hit ratio, write throughput.
- **Graceful degradation**: if KV store is unavailable, serve stale cached redirects; if CDN fails, fall back to API.

Examples

Example 1 – Generating a Slug with a Sequential Counter

```
python
import string
ALPHABET = string.digits + string.ascii_letters # 0 9 + a z + A Z (62 chars)
def encode(num):
    """Convert an integer to a base 62 string."""
    if num == 0:
        return ALPHABET[0]
    arr = []
    base = len(ALPHABET)
    while num:
        num, rem = divmod(num, base)
        arr.append(ALPHABET[rem])
    return ''.join(reversed(arr))
```

Suppose we have a distributed counter stored in DynamoDB.

```
def get_next_slug():
    counter = dynamodb.increment('global_counter') # atomic increment
    return encode(counter) # e.g., 125 !' 'cb'
---
```

***Explanation*:** The counter guarantees uniqueness. Encoding to base 62 yields a short, URL safe slug. Because the counter is atomic, no collisions occur, and the slug length grows only when the counter exceeds 62^n .

Example 2 – Read Through Cache Flow (Pseudo code)

```
```go
func redirectHandler(w http.ResponseWriter, r *http.Request) {
 slug := strings.TrimPrefix(r.URL.Path, "/")
 // 1p ã Try CDN edge cache (handled automatically by CDN)
 // 2p ã Try in process LRU cache
 if url, ok := lruCache.Get(slug); ok {
 http.Redirect(w, r, url, http.StatusMovedPermanently)
 return
 }
 // 3p ã Fallback to KV store (Redis)
 url, err := redisClient.Get(context.Background(), slug).Result()
 if err == redis.Nil {
 http.NotFound(w, r)
 return
 } else if err != nil {
 http.Error(w, "Service unavailable", http.StatusServiceUnavailable)
 return
 }
 // 4p ã Populate caches for next request
 lruCache.Add(slug, url)
 // CDN will cache the response automatically if Cache Control headers are set
 http.Redirect(w, r, url, http.StatusMovedPermanently)
}
```
```

***Explanation*:** The handler first checks an in process LRU cache (fastest). If missed, it reads from Redis (still low latency). Successful look ups are cached locally and the HTTP response includes caching headers so the CDN can store the redirect for future users, eliminating backend calls for the hottest URLs.

Quick Review

1. ****Why is a sequential counter + base 62 encoding often preferred over random tokens for a URL shortener?****
2. ****What are the three main layers of a scalable URL shortener architecture, and what role does each play?****
3. ****How does sharding by hash of the slug help prevent hot spot problems?****
4. ****Name two operational safeguards you would put in place to protect the service from abuse.****
5. ****If the primary key value store becomes unavailable, what fallback strategy can keep redirects working?****

Further Reading

- **Distributed ID Generation** – Snowflake, Sonyflake, ULID, and their trade offs.
- **Consistent Hashing & Ring Partitioning** – How to add/remove nodes without massive data movement.
- **CAP Theorem in Practice** – Choosing eventual vs. strong consistency for read heavy services.
- **CDN Edge Computing** – Using Cloudflare Workers or AWS Lambda@Edge for ultra low latency redirects.
- **Rate Limiting Algorithms** – Token bucket, leaky bucket, and distributed implementations (e.g., Redis based).

End of study notes.

